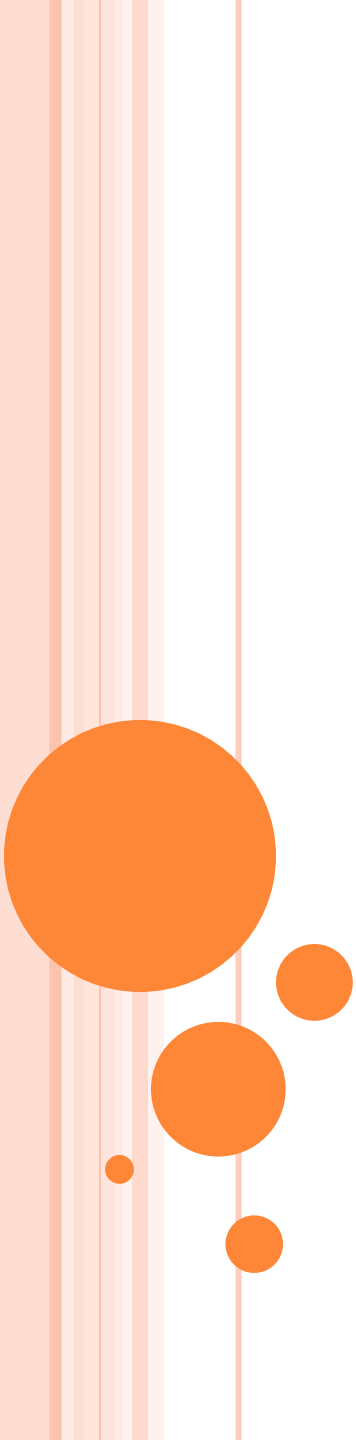




PROGRAMOWANIE GIER

WYKŁAD 2A

NOWY SYSTEM OBSŁUGI WEJŚCIA

KORUTYNY VS FUNKCJE

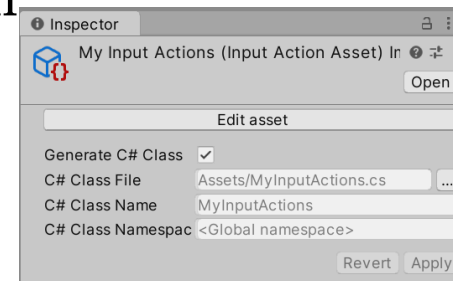
ASYNCHRONICZNE

DOSTĘPNE STRUKTURY DANYCH

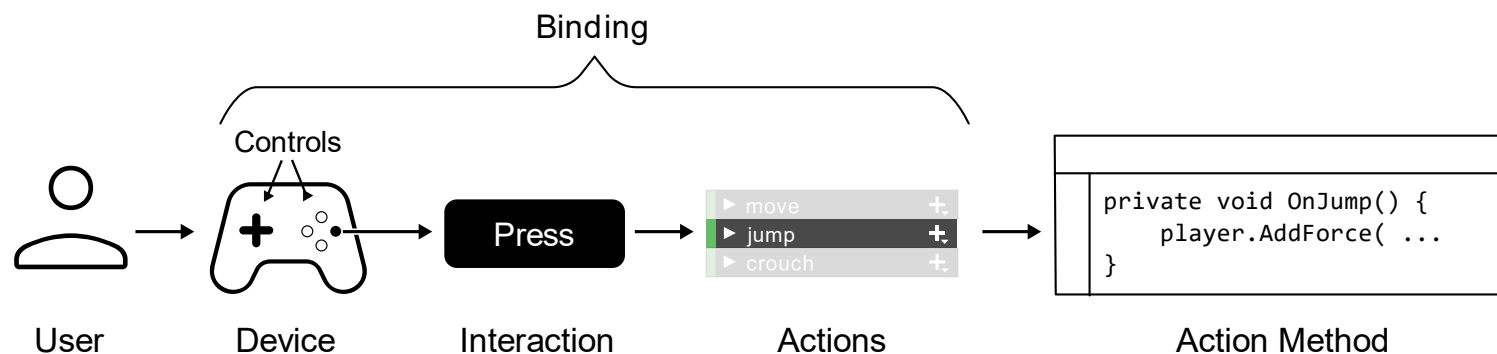
Bogumiła Hnatkowska

NOWY SYSTEM WEJŚCIA (OD VER. 2019.1)

- W wersji 2019.1 wprowadzono nowy system wejścia, który ma ułatwić współpracę z różnymi typami urządzeń
- Aktywacja nowego systemu:
 - Window/Package Manager – znaleźć pakiet Input System i zainstalować (jeżeli go nie ma)
 - Utworzyć asset reprezentujący nowy system:
Edit/Create/Input Actions
 - Wygenerować klasę reprezentującą akcje
- Tutorial: [How to use NEW Input System Package! \(Unity Tutorial - Keyboard, Mouse, Touch, Gamepad\) - Bing video](#)



NOWY SYSTEM WEJŚCIA, CONT.



- Akcje – mechanizm separujący znaczenie wejścia (skutek) od urządzenia, z którego wejście pochodzi (nie trzeba w kodzie jawnie przełączać się między urządzeniami); logiczna reprezentacja polecenia użytkownika wydanego za pomocą jednego lub wielu urządzeń
- Metoda akcji – funkcja callbackowa wywoływana w odpowiedzi na interakcję użytkownika powiązaną z daną akcją

NOWY SYSTEM WEJŚCIA, CONT.

○ Podstawowe pojęcia (klasy)

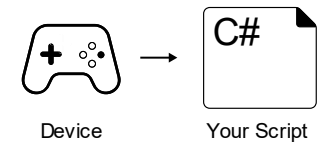
- InputAction – nazwana akcja, dla której będą wywoływane funkcje callbackowe w odpowiedzi na wejście; musi mieć unikalną nazwę i identyfikator
- InputBinding – powiązanie akcji z wejściami
- InputActionMap – kolekcja nazwanych akcji; ma unikalną nazwę i identyfikator (można po nim wyszukiwać mapę)
- InputActionAsset – kolekcja map akcji



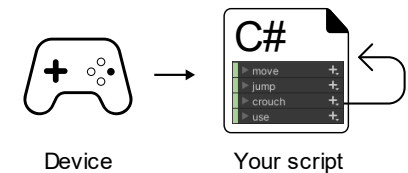
NOWY SYSTEM WEJŚCIA, CONT.

○ Scenariusze użycia nowego systemu wejścia:

- Bezpośredni odczyt danych z urządzenia



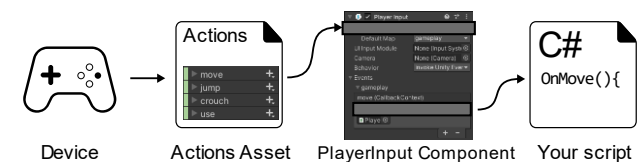
- Stosowanie akcji wbudowanych (parametry skryptu)



- Wykorzystanie assetu InputActionAsset (zbiór map akcji)



- Wykorzystanie InputAciton wraz z komponentem PlayerInput

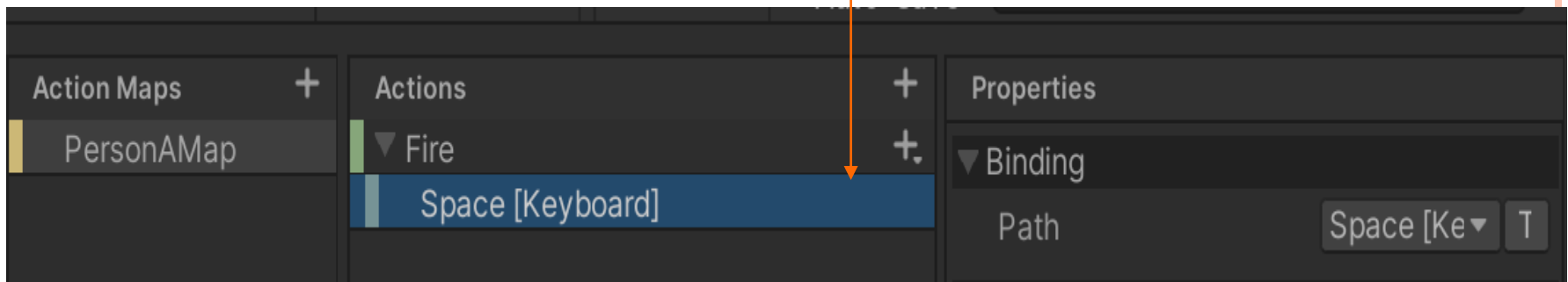
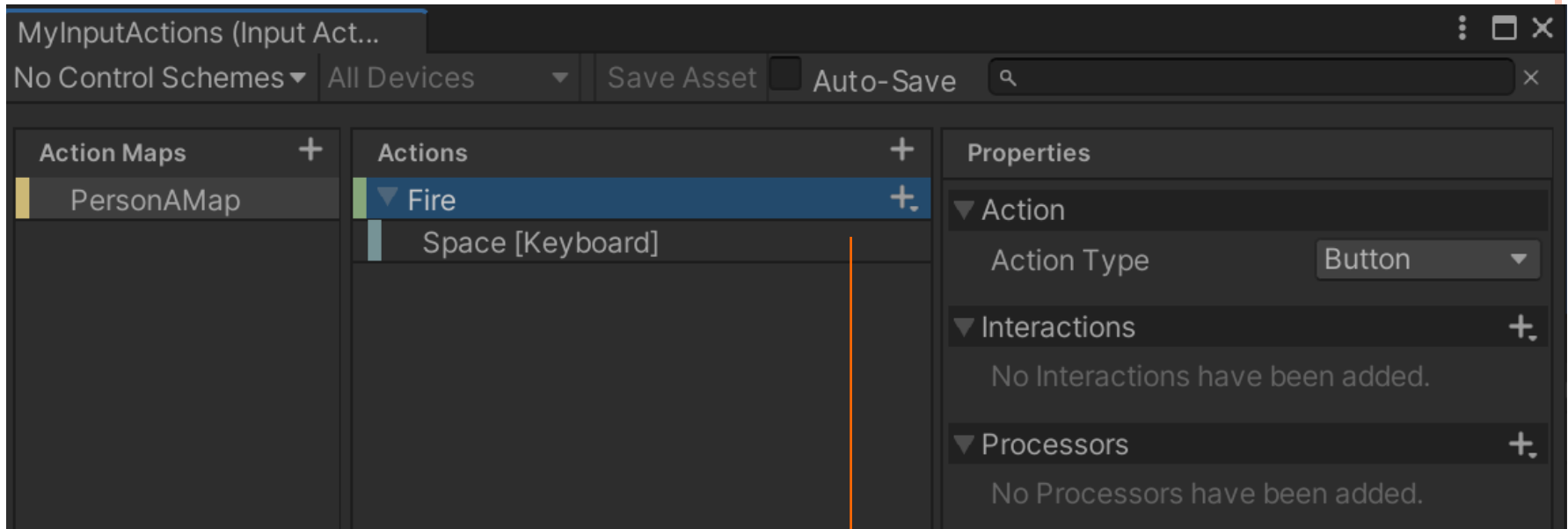


NOWY SYSTEM WEJŚCIA, CONT.

- Akcje mogą być definiowane:
 - **W specjalnym edytorze dla assetu InputAction**
 - **W skrypcie (akcje wbudowane)**
 - Ręcznie ładowane z pliku JSON
 - Ręcznie definiowane w kodzie (wszystkie elementy, w tym typ, wiązania)
- Typy akcji:
 - Value (default)
 - Button
 - Pass-Through



INPUT ACTION ASSET



OBSŁUGA AKCJI - PRZYKŁAD

```
public class ActionTester : MonoBehaviour
{
    private MyInputActions _myActions;

    private void Awake() {
        _myActions = new MyInputActions();
    }

    void OnEnable() {
        _myActions.PersonAMap.Fire.started +=
            ctx => HandleStarted();
        _myActions.PersonAMap.Fire.performed += HandleFire;
        _myActions.PersonAMap.Enable(); // włączenie konkretnej mapy
        // _myActions.Enable(); // włączenie wszystkich map
    }
}
```



OBSŁUGA AKCJI - PRZYKŁAD

```
void OnDisable() {  
    _myActions.PersonAMap.Fire.performed -= HandleFire;  
    _myActions.PersonAMap.Fire.started -= ctx=>HandleStarted();  
    _myActions.PersonAMap.Disable();  
    // _myActions.Disable();  
}  
  
private void HandleFire(InputAction.CallbackContext context) {  
    Debug.Log("Fired Action");  
}  
  
private void HandleStarted() {  
    Debug.Log("Fired Started");  
}  
}
```



DEFINICJA AKCJI (MAP AKCJI) W RAMACH MONOBEHAVIOUR

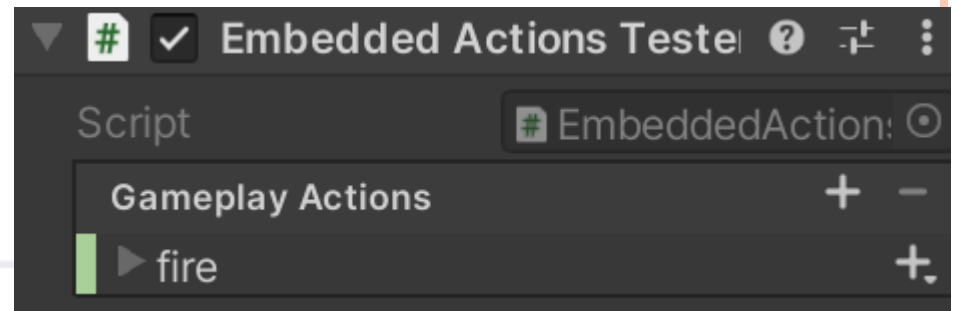
```
public class EmbeddedActionsTester : MonoBehaviour
{
    [SerializeField]
    private InputActionMap gameplayActions;

    void Awake()
    {
        gameplayActions["fire"].performed += OnFire;
    }

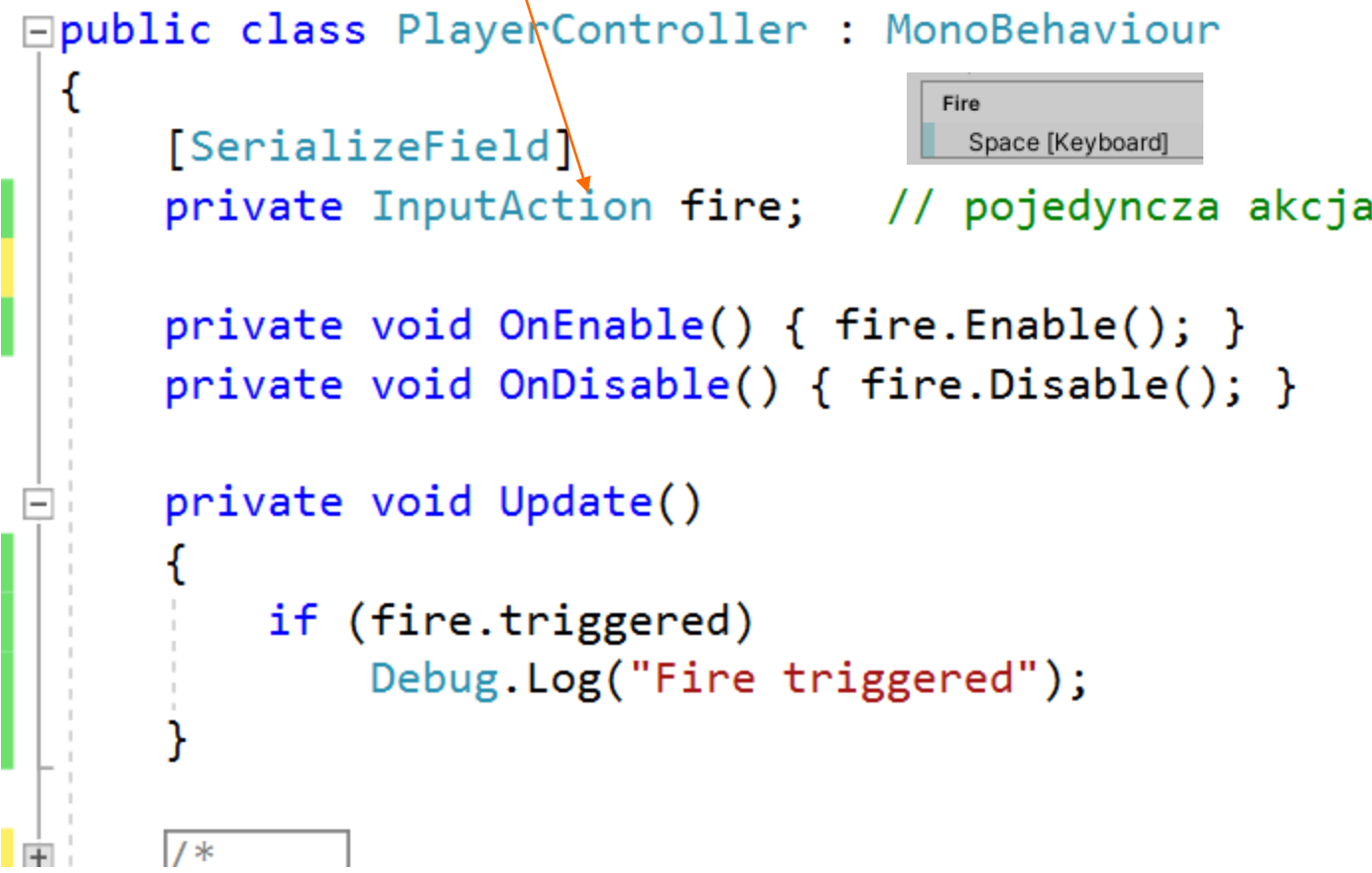
    private void OnFire(InputAction.CallbackContext obj)...

    void OnEnable()...

    void OnDisable()...
```



DEFINICJA AKCJI (MAP AKCJI) W RAMACH MONOBEHAVIOUR



```
public class PlayerController : MonoBehaviour
{
    [SerializeField]
    private InputAction fire;    // pojedyncza akcja

    private void OnEnable() { fire.Enable(); }
    private void OnDisable() { fire.Disable(); }

    private void Update()
    {
        if (fire.triggered)
            Debug.Log("Fire triggered");
    }
}
```

Fire
Space [Keyboard]

/*

KORUTYNY (COROUTINES)

- Zwykle funkcje muszą zostać wykonane w obrębie jednej ramki (nie mogą zawierać animacji, instrukcji zatrzymujących wykonanie etc.)
- *Coroutine* to funkcja, która może przerwać wykonanie, oddać sterowanie do silnika Unity (instrukcja `yield`) i być wznowiona w ramach kolejnych ramek

```
IEnumerator FunctionName() {  
    ...  
    yield return value;  
    ...  
}
```

- Przykładowa instrukcja `yield` – wznów po 1 sekundzie

```
yield return new WaitForSeconds(1.0f);
```

- Uruchomienie korutyny:

```
StartCoroutine(FunctionName());  
StartCoroutine("FunctionName");
```



KORUTYNY (COROUTINES), CONT.

- Typowe parametry yield:
 - WaitForSeconds
 - WaitForEndOfFrame
 - null (yield return null;) – wykonane w trakcie rysowania ramki (przed LateUpdate i rysowaniem sceny)
 - WaitForFixedUpdate
- Zatrzymanie korutyny:
 - StopCoroutine (wymaga podania referencji, która może być zapamiętana podczas tworzenia)
 - StopAllCoroutines
 - Deaktywacja (SetActive(false)) obiektu z korutyną, również efekt uboczny zniszczenia obiektu



FUNKCJE ASYNCHRONICZNE (ASYNC/AWAIT)

- Zadanie (Task) – reprezentacja asynchronicznej operacji. Może zwracać wynik.

```
async void DoSomething()
{
    Debug.Log("Wait for 3 seconds");
    await Task.Delay(3000);
    Debug.Log("3 seconds later");
}
```



FUNKCJE ASYNCHRONICZNE (ASYNC/AWAIT) - ZADANIA

- using System.Threading.Tasks;

```
void CountToTen()  
{  
    int number = 0;  
  
    while (number < 10)  
    {  
        number++;  
        Debug.Log(number);  
    }  
  
    Debug.Log("I've finished");  
}
```

```
async void CountToTenAsync()  
{  
    int number = 0;  
  
    while (number < 10)  
    {  
        number++;  
        Debug.Log(number);  
        await Task.Yield();  
    }  
  
    Debug.Log("I've finished");  
}
```



KORUTYNY VS FUNKCJE ASYNCHRONICZNE

Korutyny

- Nie zwracają wartości
- Wymagają udziału silnika
- Nie zachowują stosu wywołań
- Nie wspierają obsługi wyjątków
- Nie zawsze się kończą
- Cykl życia związany z obiektem typu `MonoBehaviour`

Funkcje asynchroniczne

- Mogą zwracać wartości
- Nie wymagają udziału silnika
- Zachowują stos wywołań
- Wspierają obsługę wyjątków
- Zawsze się kończą
- Cykl życia zarządzany ręcznie

[Best practices: Async vs. coroutines - Unite Copenhagen 2019 | PPT \(slideshare.net\)](#)



PODSTAWOWE TYPY KONTENERÓW

- Unity korzysta z otwartej implementacji .Net (Mono)
 - Tablice (wbudowane)
 - ArrayList
 - Tablica haszowana (HashTable)
 - Lista generyczna (Generic List)
 - Generyczny słownik (Generic Dictionary)
 - Tablica dwuwymiarowa (2D Array)



PODSTAWOWE TYPY KONTENERÓW – TABLICE WBUDOWANE

```
TheType myArray = new TheType[lengthOfArray];  
int[] myNumbers = new int[10];  
GameObject[] enemies = new GameObject[16];  
int howBig = myArray.Length;  
myArray[i] = newValue;  
thisValue = myArray[i];  
string[] weapons =  
    new string[] { "Sword", "Knife", "Gun" };
```



PODSTAWOWE TYPY KONTENERÓW – TABLICE WIELOWYMIAROWE

```
string[,] myArray = new string[16,4];  
Tile[,] map = new Tile[32,32];  
myArray[x,y] = newValue;  
ValueType thisValue = myArray[x,y];  
int width = myArray.GetLength(0);  
int length = myArray.GetLength(1);
```



PODSTAWOWE TYPY KONTENERÓW – GENERICLIST

```
using System.Collections.Generic;
```

```
List<Type> myList = new List<Type>();
```

```
List<int> someNumbers = new List<int>();
```

```
List<GameObject> enemies = new List<GameObject>();
```

```
myList.Add(theItem);
```

```
myList[i] = newItem;
```

```
Type thisItem = myList[i];
```

```
myList.RemoveAt(i);
```



PODSTAWOWE TYPY KONTENERÓW – DICTIONARY

```
using System.Collections.Generic;
```

```
Dictionary<KeyType,ValueType> myDictionary =  
    new Dictionary<KeyType,ValueType>();
```

```
Dictionary<string,Person> myContacts =  
    new Dictionary<string,Person>();
```

```
myDictionary[anyKey] = newValue;
```

```
thisValue = myDictionary[theKey];
```

```
int howBig = myDictionary.Count;
```

```
myDictionary.Remove(theKey);
```



PODSTAWOWE TYPY KONTENERÓW – ARRAYLIST

```
ArrayList myArrayList = new ArrayList();  
myArrayList.Add(anItem);  
myArrayList[i] = newValue;  
TheType thisItem = (TheType) myArray[i];  
myArray.RemoveAt(i);  
var howBig = myArray.Count;
```



PODSTAWOWE TYPY KONTENERÓW – HASHTABLE

```
Hashtable myHashtable = new Hashtable();  
myHashtable[anyKey] = newValue;  
ValueType thisValue =  
    (ValueType)myHashtable[theKey];  
int howBig = myHashtable.Count;  
myHashtable.Remove(theKey);
```



SERIALIZACJA W UNITY

- Proces automatycznego przekształcania struktur danych lub stanu obiektu do formatu zarządzanego przez Unity (który może być zapisany i odczytany)
- Mechanizm stosowany do zapisywania i ładowania scen i asetów na dysku, czy w play mode
- Mechanizm stosowany w oknie Inspektora



SERIALIZACJA W UNITY

○ Reguły serializacji:

- Serializowanie pól w skryptach: publiczne lub z atrybutem [SerializeField], nie są statyczne, nie są tylko do odczytu lub stałe; mają typ danych, który może być zserializowany
- Typy, które mogą być serializowane:
 - Typy proste (int, float, double, ...)
 - Pewne wbudowane typy Unity, np. VectorX, Rect, Quaternion, Matrix4x4, Color, Color32
 - Typy wyliczeniowe
 - Referencje do obiektów, które dziedziczą po UnityEngine.Object; ich klasy powinny być oznaczone atrybutem [Serializable], nie być abstrakcyjne, nie być statyczne, nie generyczne (ale mogą z nich dziedziczyć)
 - Struktury posiadające atrybut [Serializable]
 - Wybrane kontenery: tablice typów prostych, List<T>
 - Uwaga: brak wsparcia dla struktur wielowymiarowych
- Brak wsparcia dla polimorfizmu
- Brak wsparcia dla pól null
- Pola, których nie chcesz serializować zaznacz [NonSerialized]

○ Zalecenia:

- Serializuj tylko niezbędne dane
- Nie serializuj duplikatów danych – trudno utrzymać synchronizację
- Staraj się nie serializować złożonych, zagnieżdżonych struktur

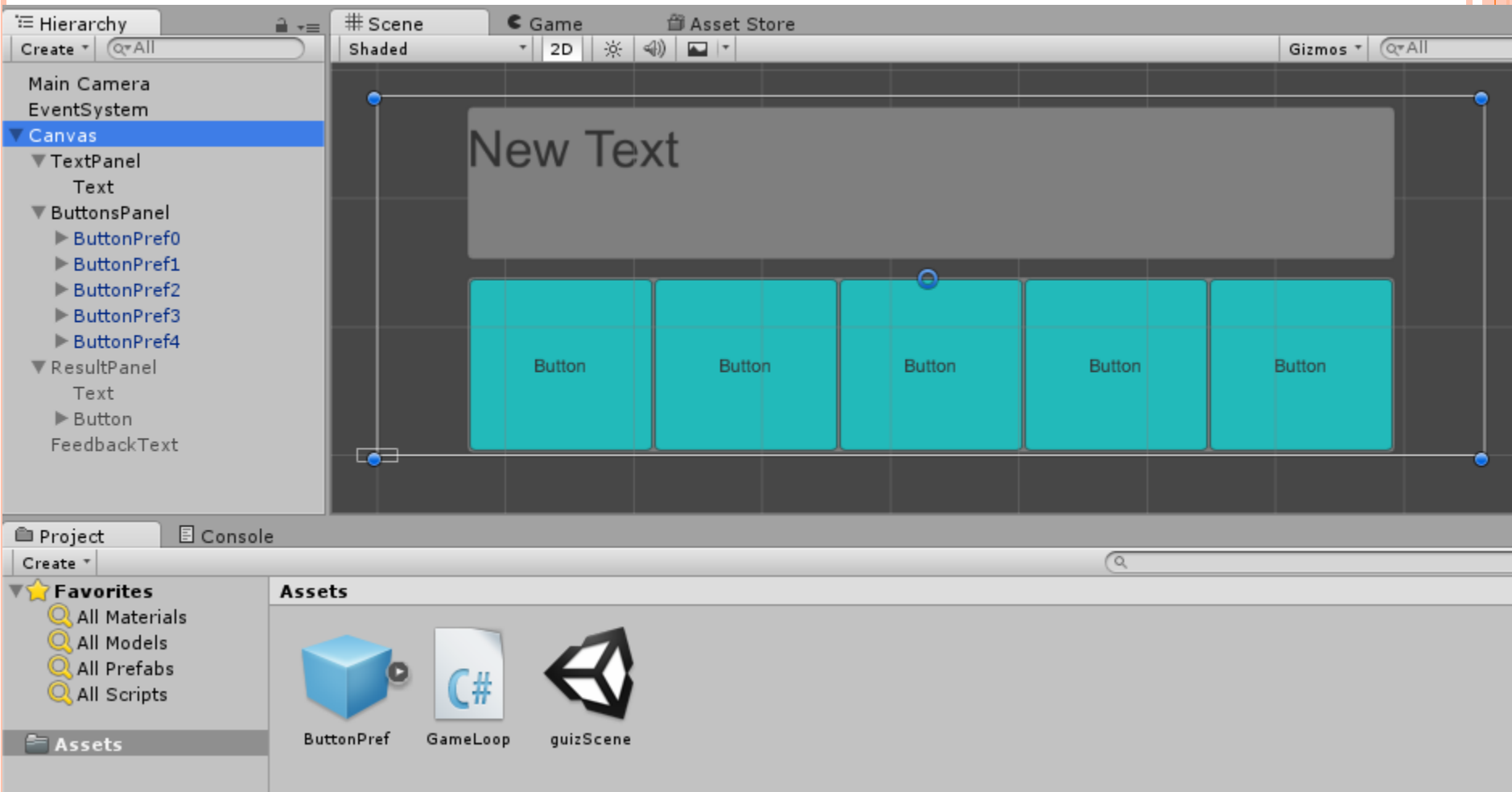


PIERWSZA GRA - QUIZ

- Gra 2D wykorzystująca elementy UI:
 - Quiz składający się z 3 pytań losowanych z „bazy”:
 - Nagłówek z pytaniem
 - 5 odpowiedzi (tylko 1 poprawna)
 - Informacja zwrotna po każdym pytaniu
 - Informacja zwrotna na koniec quizu
 - Możliwość powtórzenia quizu



ELEMENTY GRAFICZNE GRY



PRYWATNE STRUKTURY DANYCH

```
private struct Question
{
    public string questionText;
    public string[] answers;
    public int correctAnswerIdx;

    public Question(string questionText, string[] answers, int correctAnswer) {
        this.questionText = questionText;
        this.answers = answers;
        this.correctAnswerIdx = correctAnswer;
    }
}

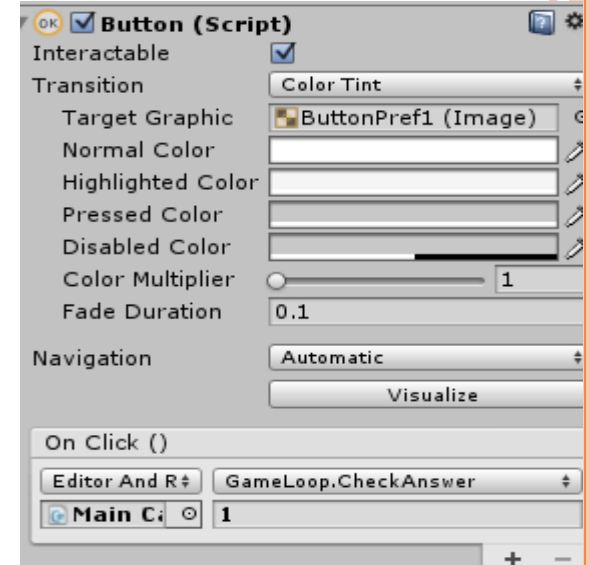
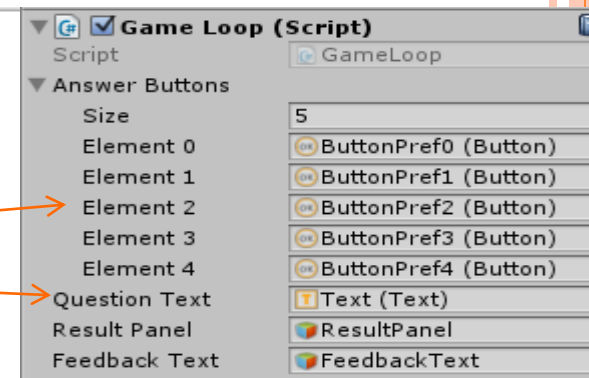
private Question currentQuestion;    // currently asked question
private int currentQuestionIdx;      // index of currently asked question
private Question[] questionSet = new Question[5];    // question database
private ArrayList questionNumbersSelected = new ArrayList();    // list of asked question idx
private int questionNumber = 3;      // number of question to asked
private int noOfCorrectAnswers = 0;  // number of correct answers
```



POWIĄZANIE PUBLICZNYCH ATRYBUTÓW SKRYPTU Z OBIEKTAMI GRY W OKNIE INSPEKTORA

○ Skrypt z logiką gry

```
1 using UnityEngine;
2 using System.Collections;
3 using UnityEngine.UI;
4 using UnityEngine.SceneManagement;
5
6 public class GameLoop : MonoBehaviour {
7     public Button[] answerButtons;    // UI Buttons
8     public Text questionText;        // UI Text with question
9     public GameObject resultPanel;    // UI Result Panel (inactive at the beginning)
10    public GameObject feedbackText;   // UI feedback text (inactive at the beginning)
11
12    // Funkcja uruchamiana w efekcie zdarzenia naciśnięcia przycisku
13    public void CheckAnswer(int buttonNumber) {
14        Text text;
15        text = feedbackText.GetComponent ("Text") as Text;
16
17        if (buttonNumber == currentQuestion.correctAnswerIdx) {
18            Debug.Log("Correct");
19            text.text = "Dobrze";
20            text.color = Color.green;
21            noOfCorrectAnswers++;
22        } else {
23            Debug.Log("Incorrect");
24            text.text = "Źle";
25            text.color = Color.red;
26        }
27    }
28
29    StartCoroutine("DelayMoveToNextQuestion");
30 }
```



WYBRANE FUNKCJE

// Aktywuje panel z informacją zwrotną, a p 1 s. go deaktywuje

```
IEnumerator DelayMoveToNextQuestion(){  
    feedbackText.SetActive (true);  
    yield return new WaitForSeconds (1.0f);  
    feedbackText.SetActive (false);  
    MoveToNextQuestion ();  
}
```

// Przygotowanie wyników i ich wyświetlenie

```
void FinishGame(){  
    GameObject panel = GameObject.Find ("TextPanel");  
    panel.SetActive (false);  
  
    panel = GameObject.Find ("ButtonsPanel");  
    panel.SetActive (false);  
  
    resultPanel.GetComponentInChildren<Text> ().text = "Twój wynik : " + noOfCorrectAnswers + " z 3";  
    resultPanel.SetActive (true);  
}
```

// Obsługa przycisku restartu gry

```
public void RestartGame(){  
    Scene scene = SceneManager.GetActiveScene();  
    SceneManager.LoadScene(scene.name);  
}
```



PYTANIA KONTROLNE

- Jakimi terminami posługuje się tzw. nowy system wejścia?
- Jak zrealizować korutynę?
- Jak uruchomić funkcję asynchroniczną?
- Jakie są podstawowe różnice między korutyną i funkcją asynchroniczną?

